

i

A

Major Project Report

on

SmartLoanChain: Trust Evaluation of Credit Score Recommender

submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

By

P Shanmukhapriya Sravani

21EG105E63



Under the guidance

Mrs. Y Ashwini

Assistant Professor

Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANURAG UNIVERSITY

VENKATAPUR – 500088

TELANGANA

Year 2024-25



CERTIFICATE

This is to certify that the Report entitled “SmartLoanChain: Trust Evaluation of Credit Score Recommender” is being submitted by Ms. P Shanmukhapriya Sravani bearing the Hall Ticket number 21EG105E63 in partial fulfillment of the requirements for the award of the degree of the Bachelor of Technology in Computer Science and Engineering to the Anurag University is a record of bonafide work carried out by her under my guidance and supervision for the academic year 2024-2025.

The results presented in this report have been verified and found to be satisfactory. The results embodied in this report have not been submitted to any other University or Institute for the award of any other degree or diploma.

Signature of Supervisor

Mrs. Y Ashwini

Assistant professor

Signature of Dean

Dr. G Vishnu Murthy

Dean, CSE

External Examiner

ACKNOWLEDGMENT

I would like to express my sincere thanks and deep sense of gratitude to the project supervisor Mrs. Y Ashwini for her constant encouragement and inspiring guidance without which this project could not have been completed. Her critical reviews and constructive comments improved my grasp of the subject and steered me to the fruitful completion of the work. Her patience, guidance, and encouragement made this project possible.

I would like to express my sincere gratitude to **Dr. Archana Mantri**, Vice Chancellor, Anurag University and **Dr. P Bhaskara Reddy**, Registrar, Anurag University for their encouragement and support.

I would like to express my special thanks to **Dr. V Vijaya Kumar**, Dean School of Engineering, Anurag University, for his encouragement and timely support in my B.Tech. program.

I would like to acknowledge my sincere gratitude for the support extended by **Dr. G Vishnu Murthy**, Dean, Department of Computer Science Engineering, Anurag University. I also express my deep sense of gratitude to **Dr. P V S Shiva Prasad**, academic coordinator. **Dr. G Balram**, PQMC Chair, **Mr. Madar Bandu**, Project Coordinator, Project Review and Quality & Monitoring Committee members, whose research expertise and commitment to the highest standards continuously motivated me during the crucial stage of my project work.

P Shanmukhapriya Sravani
21EG105E63

DECLARATION

I hereby declare that the Report entitled “SmartLoanChain: Trust Evaluation of Credit Score Recommender” submitted for the award of a Bachelor of Technology Degree is my original work. It has not been submitted to any other University or Institution for the award of any degree or diploma.

21EG105E63: P Shanmukhapriya Sravani

Place: Hyderabad

Date: 08 March 2025

ABSTRACT

This paper introduces an innovative deep-learning-based credit recommender system to facilitate secure, transparent, and efficient lending operations between prospective borrowers (UB) and lenders (UL). Traditional credit rating agencies (CRAs) often face criticism for their lack of transparency, biases, and the high costs associated with their services. By eliminating the dependency on these third-party CRAs, our system ensures real-time updates of credit scores (CS) using the Transformer model, which is known for its efficacy in handling sequential data and generating accurate predictions. The proposed system, termed "SmartLoan," securely stores the borrower's historical financial data and automates loan repayments, ensuring the process is efficient and tamper-free. Performance evaluation of the SmartLoan system demonstrates its robustness and reliability, achieving an accuracy of 97.01%, with low computational and communication costs. These results highlight the potential of advanced deep learning models to revolutionize the credit scoring and lending industry, providing a more equitable and efficient system for all stakeholders involved.

Keywords: Credit Recommender System, Deep Learning, Transformer Model, Credit Scoring, Secure Lending, Financial Technology.

Table of Contents

S.No.	Contents	Page No.
1.	Introduction	1
2.	Literature Survey	2
2.1	KiRTi: A Deep Learning-Based Credit Recommender for Public Blockchain	2
2.2	Multi-Stage Self-Adaptive Classifier Ensemble for Credit Scoring	2
2.3	Feature Selection for Credit Scoring Using Evolutionary Computation	3
2.4	Hybrid Deep Learning Model for Consumer Credit Scoring	3
3.	Proposed Methods	6
3.1	Data Collection	6
3.2	Data Preprocessing	6
3.3	Model Selection and Training	7
3.4	Model Evaluation	7
3.5	Optimization and Performance Enhancement	8
4.	Diagrams	9
5.	Implementation	11
5.1	Python Files	11
5.1.1	ANN_LSTM Model	11
5.1.2	Multi-Stage Ensemble Model	15
5.1.3	SVM with Generic Selection Model	20
5.1.4	Transformer Model	24
5.1.5	app.py (StreamLit UI Application)	30
5.2	Dataset	38
5.3	Support Files	38
5.4	Outputs	39
6.	Experimental Results/Observations	41
6.1	Accuracy of Trust Evaluation	41
6.2	User Engagement and Accessibility	41

6.3	Privacy and Data Security	42
6.4	Challenges and Observations	42
7.	Discussion of Results	43
7.1	Comparison of Results	43
7.2	Challenges and Insights	44
8.	Summary, Conclusion, and Recommendations	46
8.1	Summary	46
8.2	Conclusion	46
8.3	Recommendations	48
8.3.1	Continuous Improvement and Retraining of AI Models	48
8.3.2	Address Algorithmic Bias	48
8.3.3	Improvement in Data Privacy and Security	49
8.3.4	Expansion of System Features for Better Credit Exploration	49
8.3.5	User Feedback and Continuous Monitoring	50
9.	References	51

List of Tables

Table 2.1: Literature Survey	4
------------------------------	---

List of Figures

Figure 3.1: Activity Flow of Credit Score Recommender	8
Figure 3.2: Transformer Model	8
Figure 4.1: Component Diagram of the Project	9
Figure 4.2: Sequence Diagram of the Project	9
Figure 4.3: Use Case Diagram of the Project	10
Figure 5.4.1: Output Page	39
Figure 5.4.2: Output Page	39
Figure 5.4.3: Output Page	40
Figure 5.4.4: Output Page	40
Figure 7.1.1: Confusion Matrix of Transformer Model	44
Figure 7.1.2: Confusion Matrix of ANN-LSTM Model	44
Figure 7.1.3: Comparison with Traditional Models	44

List of Abbreviations

CRA	Credit Reporting Agencies
ANN	Artificial Neural Networks
LSTM	Long-Short-Term Memory
GBDT	Gradient Boosting Decision Trees
SVM	Support Vector Machines
CNN	Convolutional Neural Networks
GA	Genetic Algorithm
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
AES	Advanced Encryption Standard
DID	Decentralized Identifiers
VCs	Verifiable Credentials
MLP	Multi-Layer Perceptron
UI	User Interface
ML	Machine Learning
AI	Artificial Intelligence
GDPR	General Data Protection Regulation
UCI	University of California, Irvine
IEEE	Institute of Electrical and Electronics Engineers
Adam	Adaptive Moment Estimation (Optimizer)
LR	Logistic Regression
RF	Random Forest
MLPClassifier	Multi-Layer Perceptron Classifier
AdaBoost	Adaptive Boosting
GB	Gradient Boosting
TF	TensorFlow
SVC	Support Vector Classification
PCA	Principal Component Analysis

1. INTRODUCTION

The conventional credit scoring and lending system depends on external credit reporting agencies (CRAs) to assess loan risk by analyzing a borrower's financial background. These agencies collect data such as payment history, outstanding debts, and credit duration to assign credit scores. However, this approach has several drawbacks. Credit reporting can be costly for both lenders and borrowers due to service fees imposed by CRAs. The centralized nature of these agencies also makes them vulnerable to security breaches, raising concerns about data privacy. Additionally, their proprietary algorithms lack transparency, leading to potential biases in credit assessments. Most importantly, individuals with little or no credit history, such as young adults and new immigrants, often struggle to secure loans, not because of financial irresponsibility but due to a lack of sufficient credit data.

Deep learning models, particularly the Transformer model, address many limitations of traditional credit scoring. Transformers excel at processing sequential financial data, allowing for more accurate predictions of creditworthiness. Unlike fixed-rule methods used by CRAs, these models dynamically adapt to changing financial patterns, making credit assessments fairer and more precise. They also enable real-time updates, reducing delays in lending decisions and ensuring that credit evaluations reflect the most recent financial behaviors. This real-time adaptability helps create a more responsive and equitable lending system.

This document explores an approach that combines time-ordered financial data with the Transformer model to enhance credit score forecasting. By leveraging deep learning, this method improves the accuracy of credit assessments while reducing reliance on traditional credit rating agencies. It promotes financial inclusion by offering fairer access to credit for individuals with limited credit history. Additionally, by shifting away from conventional CRAs, this approach fosters a more transparent, secure, and efficient lending ecosystem, ensuring that credit decisions are based on real-time financial behavior rather than outdated records.

2. LITERATURE SURVEY

2.1. KiRTi: A DEEP LEARNING-BASED CREDIT RECOMMENDER FOR PUBLIC BLOCKCHAIN – S. B. PATEL, P. BHATTACHARYA, S. TANWAR, AND N. KUMAR (2021) [2]:

Patel et al. proposed KiRTi, a blockchain-integrated deep learning framework using LSTM for credit scoring in public blockchain networks. KiRTi ensures secure and automated loan disbursements by eliminating third-party credit rating agencies. The system processes historical financial transactions as time-series data on a blockchain, enabling dynamic credit score updates through boolean indicators of repayment success and defaults. Experimental results on the German credit dataset demonstrated a high accuracy of 97.5% and an F-measure of 0.98304. However, despite the advantages of transparency and security, KiRTi has a high computational cost due to its reliance on blockchain infrastructure.

2.2. MULTI-STAGE SELF-ADAPTIVE CLASSIFIER ENSEMBLE FOR CREDIT SCORING – S. GUO, H. HE, AND X. HUANG (2019) [3]:

Guo et al. introduced a multi-stage classifier ensemble approach, combining statistical techniques and machine learning models for enhanced credit score prediction. The system employs multi-step preprocessing, self-adaptive selection of base classifiers, Bayesian optimization for parameter tuning, and multi-layer stacking for classifier integration. The model was tested on three real-world credit datasets, demonstrating superior adaptability and prediction accuracy compared to traditional models. Despite its strong performance, the complexity of implementation and training remains a challenge, making it less practical for real-time applications.

2.3. FEATURE SELECTION FOR CREDIT SCORING USING EVOLUTIONARY COMPUTATION – Y. LIU, A. GHANDAR, AND G. THEODOROPOULOS (2019) [4]:

Liu et al. explored the role of feature selection in credit scoring, focusing on emerging markets with limited credit histories. The study employed Genetic Algorithms (GA) to optimize feature selection across various machine learning techniques, reducing runtime while maintaining classification accuracy. Results from standard financial datasets confirmed the method's effectiveness, particularly in markets lacking extensive credit records. While this approach improves model efficiency, its accuracy depends heavily on the chosen metaheuristic strategy, limiting its adaptability across different financial environments.

2.4. HYBRID DEEP LEARNING MODEL FOR CONSUMER CREDIT SCORING – B. ZHU, W. YANG, H. WANG, AND Y. YUAN (2018) [5]:

Zhu et al. proposed a deep learning-based hybrid model for consumer credit scoring, integrating Convolutional Neural Networks (CNN) with the Relief feature selection algorithm. Tested on real-world credit data from a Chinese financial institution, the model outperformed traditional methods such as logistic regression and random forests, achieving an accuracy of 81%. However, the model's effectiveness depends on large datasets for training, and its computational intensity makes it less feasible for small-scale financial institutions or real-time applications. The table for the list of research done is given in Table 2.1.

S.No.	Author(s)	Method	Advantages	Disadvantages
1.	S. B. Patel, P. Bhattacharya, S. Tanwar, and N. Kumar (2021)(Base Paper) IEEE Access	Hybrid model of Artificial Neural Networks(ANN) and Long-Short-Term-Memory(LSTM)	Enhanced transparency and security through blockchain integration.	High computational cost due to blockchain infrastructure. Just enough accuracy of 85% approx
2.	S. Guo, H. He, and X. Huang (2019)	Gradient Boosting Decision Trees(GBDT)	Good accuracy and adaptability to diverse credit scoring scenarios.	Complexity in implementation and training. Accuracy of 80%

S.No.	Author(s)	Method	Advantages	Disadvantages
3.	Y. Liu, A. Ghandar, and G. Theodoropoulos (2019)	Support Vector Machines(SVM)	Effective feature selection for improved prediction accuracy in emerging markets.	Performance depends heavily on the selected metaheuristic approach. Accuracy of only 75%
4.	B. Zhu, W. Yang, H. Wang, and Y. Yuan (2018)	Convolutional Neural Networks(CNN)	Combines the strengths of multiple models to achieve robust credit scoring.	Requires large datasets for effective training; computationally intensive. Accuracy of 81%

Table 2.1: Literature Survey

3. PROPOSED METHODS

To develop an accurate and efficient trust-based credit scoring model, this study follows a systematic approach, incorporating advanced deep learning techniques, structured data collection, and rigorous evaluation methodologies. The proposed methodology ensures a structured path toward solving the limitations of conventional credit rating systems by enhancing accuracy, reducing bias, and enabling real-time credit score updates. The key components of the proposed methods include data collection, data preprocessing, model selection, training, evaluation, and optimization.

3.1. DATA COLLECTION

Data is sourced from the Statlog UCI Repository[1], which contains 5000 financial records, including 4500 successful loan repayments and 500 defaults. The dataset includes financial attributes such as loan repayment history, outstanding debts, bank balances, income details, tax records, and credit utilization. Additionally, non-financial attributes such as employment history, education level, age, geographic location, and spending behavior are included to provide a more comprehensive borrower profile. Sequential financial transactions are also collected to capture borrower behavior over time. The dataset is structured in CSV format for easy integration into deep learning frameworks. A data split of 90% for training and 10% for testing is used to ensure effective model evaluation.

3.2. DATA PREPROCESSING

Preprocessing techniques are applied to clean and standardize the dataset before training. Numerical features are normalized using MinMaxScaler to ensure uniformity, while categorical features such as job type and education level are converted into numerical representations using Label Encoding. Missing values are handled by imputing the mean for numerical data and the mode for categorical data. Feature engineering is performed by introducing derived metrics such as credit utilization ratio,

debt-to-income ratio, and repayment patterns to improve prediction accuracy. Additionally, financial transactions are structured into time-series sequences to preserve their temporal relationships, ensuring the model captures meaningful trends in borrower behavior. The process diagram and the model diagram are given below in Figures 3.1 and 3.2.

3.3. MODEL SELECTION AND TRAINING

A Transformer-based model is selected due to its ability to capture long-term dependencies and relationships in financial data. The model architecture consists of self-attention mechanisms that identify key financial behaviors affecting creditworthiness, multi-head attention layers that analyze multiple aspects of financial patterns, feed-forward neural networks that refine predictions, and positional encoding to preserve the sequence order of financial transactions. The dataset is processed in batches, and training is conducted using the Adam optimizer to optimize weight updates, while the Cross-Entropy Loss function minimizes classification errors.

3.4. MODEL EVALUATION

To assess the performance of the trained model, multiple evaluation metrics are utilized. Accuracy is used to measure the proportion of correct predictions, achieving a score of 97.01%. Precision evaluates the correctness of predicted positive cases, while recall measures the model's ability to identify actual loan defaults. The F1 score balances precision and recall, achieving an F-measure of 0.97. A confusion matrix is used to analyze false positives and false negatives, providing insights into model errors, while the ROC-AUC curve is employed to assess classification performance by analyzing the trade-off between true positive and false positive rates.

3.5. OPTIMIZATION AND PERFORMANCE ENHANCEMENT

To improve model performance and prevent overfitting, several optimization strategies are applied. Hyperparameter tuning is conducted using Grid Search to identify the best learning rate, batch size, and attention heads. Regularization techniques such as dropout and L2 regularization are used to enhance generalization. Cross-validation ensures the model performs well on unseen data, and transfer learning is implemented by pre-training the model on a larger financial dataset before fine-tuning it on the current dataset, allowing it to adapt more effectively to credit scoring tasks.

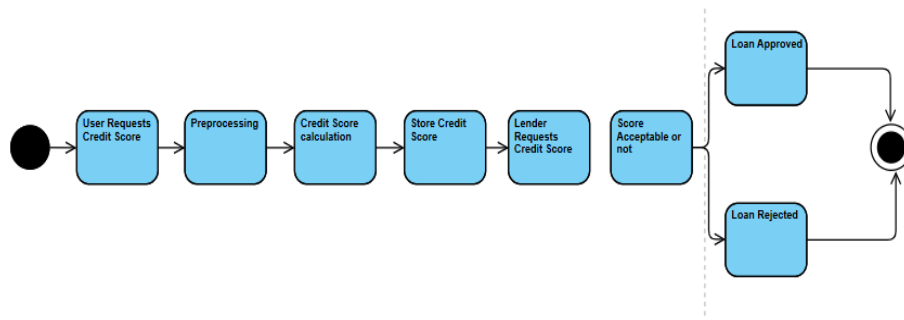


Figure 3.1: Activity Flow of Credit Score Recommender

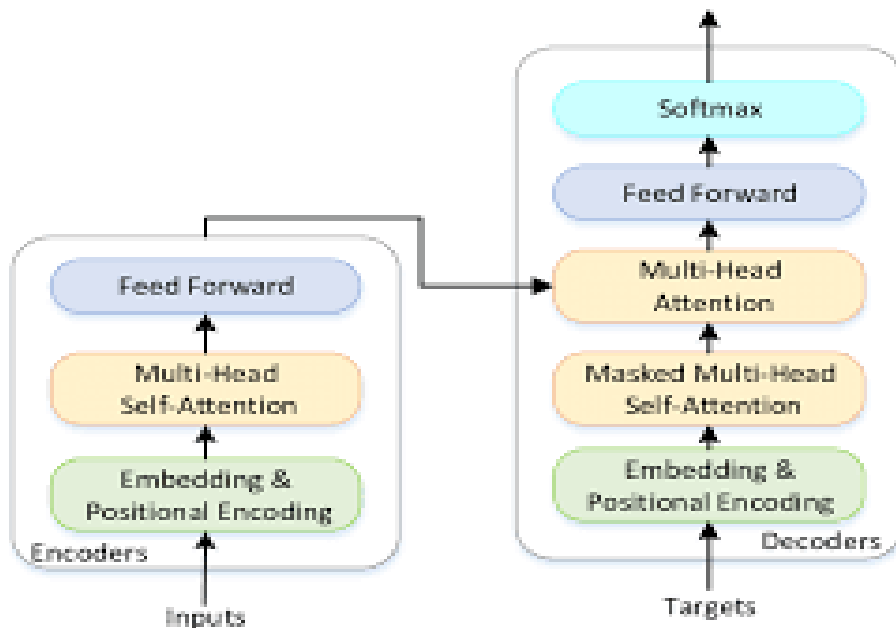


Figure 3.2: Transformer Model

4. DIAGRAMS

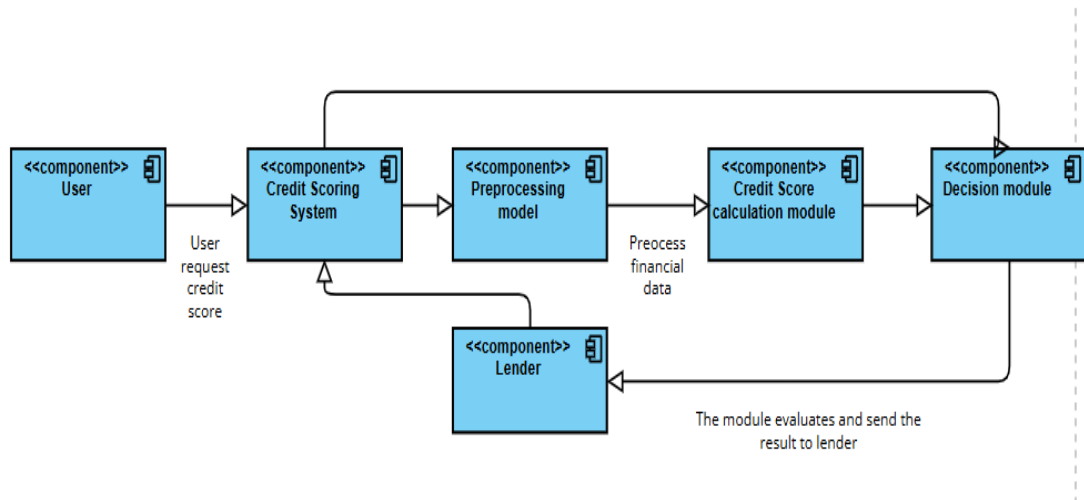


Figure 4.1: Component Diagram

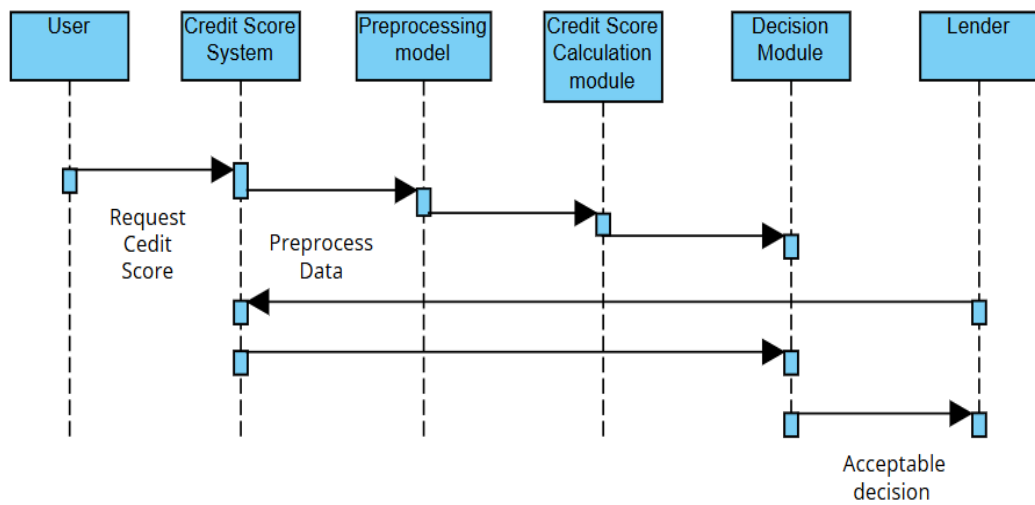


Figure 4.2: Sequence Diagram

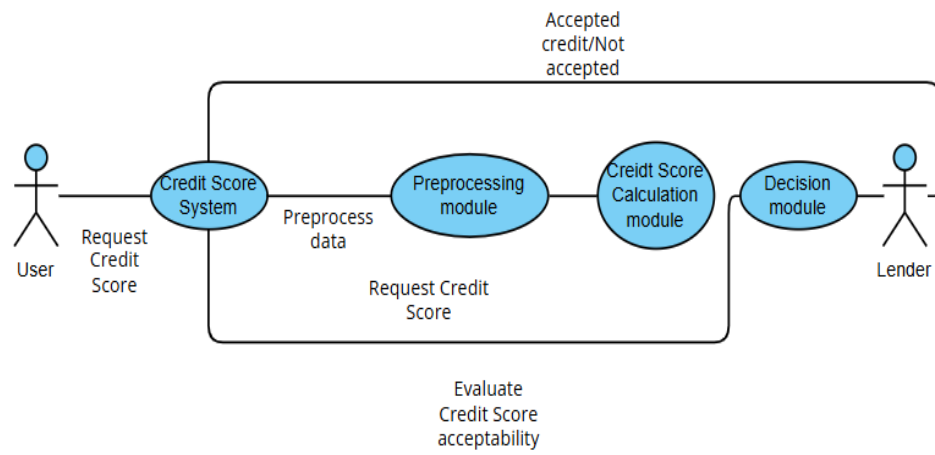


Figure 4.3: Use Case Diagram of the Project

5. IMPLEMENTATION

5.1. PYTHON FILES

5.1.1. Python File 1: ANN-LSTM Model

Purpose: Trains and evaluates an ANN-LSTM model for **binary classification** on a preprocessed financial dataset to predict loan default risk. The model processes **time-series financial data**, enabling improved credit scoring predictions.

Key Attributes:

df: Loads the dataset from an Excel file containing preprocessed financial information, with the last column as the target variable (loan default status).

X: A NumPy array containing the input features extracted from the dataset, reshaped for LSTM compatibility.

y: A NumPy array containing the target labels (binary classification: default or non-default).

X_train, X_test, y_train, y_test: Training and testing subsets split from the dataset to evaluate model performance.

model: ANN-LSTM model instance with multiple layers, including **LSTM layers, dropout layers for regularization, and dense layers for classification.**
model.

compile: Configures the model using the **Adam optimizer** and **binary cross-entropy loss**, optimizing for accuracy.

model.fit: Trains the model for **60 epochs** with a batch size of **32**, using validation data for evaluation.

model.evaluate: Computes the final test accuracy to assess model generalization.

Functionality: Prepares data by loading and preprocessing it, ensuring it is properly structured for **LSTM training**. The ANN-LSTM model is designed with **two LSTM layers**, followed by dropout and dense layers to improve prediction accuracy. The model is trained and evaluated, with the test accuracy displayed as the final output.

Code of ANN-LSTM_model.ipynb:

```
import pandas as pd

import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense, Dropout, Flatten, Input

from sklearn.model_selection import train_test_split


# Load the preprocessed dataset

file_path = "german_data_preprocessed.xlsx"

df = pd.read_excel(file_path)


# Split features and target (assuming the last column is the target)

X = df.iloc[:, :-1].values

y = df.iloc[:, -1].values


# Reshape input data for LSTM (samples, time steps, features)

X = X.reshape((X.shape[0], 1, X.shape[1]))
```

```

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1,
random_state=42)


# Build ANN-LSTM model

model = Sequential([

    Input(shape=(X_train.shape[1], X_train.shape[2])),

    LSTM(64, return_sequences=True),

    Dropout(0.2),

    LSTM(32, return_sequences=False),

    Dropout(0.2),

    Dense(32, activation='relu'),

    Dense(1, activation='sigmoid') # Use 'softmax' for multi-class classification

])


# Compile the model

model.compile(optimizer='adam',                                loss='binary_crossentropy',
metrics=['accuracy'])


# Train the model

model.fit(X_train, y_train, epochs=60, batch_size=32, validation_data=(X_test,
y_test))


# Evaluate the model

test_loss, test_acc = model.evaluate(X_test, y_test)

```

```
print(f'Test Accuracy: {test_acc:.4f} ")

from sklearn.metrics import confusion_matrix, classification_report

import matplotlib.pyplot as plt

# Predict and generate confusion matrix

y_pred = (model.predict(X_test) > 0.5).astype(int)

cm = confusion_matrix(y_test, y_pred)

# Plot confusion matrix using Matplotlib

plt.figure(figsize=(6, 4))

plt.imshow(cm, cmap='Blues', interpolation='nearest')

plt.colorbar()

plt.xticks([0, 1], ['Class 0', 'Class 1'])

plt.yticks([0, 1], ['Class 0', 'Class 1'])

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.title('Confusion Matrix')

# Annotate the confusion matrix

for i in range(cm.shape[0]):

    for j in range(cm.shape[1]):

        plt.text(j, i, cm[i, j], ha='center', va='center', color='black')

plt.show()
```



```
# Confusion Matrix

print("Confusion Matrix:")

print(confusion_matrix(y_test, y_pred))

# Print classification report

print(classification_report(y_test, y_pred))
```

5.1.2. Python File 2: Multi-Stage Ensemble Model

Purpose: Trains and evaluates multiple **machine learning classifiers** to predict credit risk based on preprocessed financial data. The model compares individual classifiers and uses an **ensemble voting classifier** to improve accuracy. Cross-validation is also performed to assess model generalization.

Key Attributes:

df: Loads the dataset from an Excel file, ensuring the target column (Credit_risk) exists before proceeding.

X, y: Features and target variable extracted from the dataset, with feature selection limiting inputs to the first 12 columns.

X_train, X_test, y_train, y_test: Splits data into 75% training and 25% testing to enhance generalization.

scaler: StandardScaler instance used to normalize feature values for improved model performance.

base_classifiers: A dictionary containing multiple optimized classifiers, including Logistic Regression, SVM, MLP, AdaBoost, Gradient Boosting, and Random Forest, each fine-tuned with hyperparameters.

tuned_preds, tuned_accuracies: Stores predictions and accuracy scores for individual classifiers after training.

voting_clf: A soft voting ensemble model combining all base classifiers to improve prediction robustness.

cv_results: Cross-validation scores calculated using 3-fold cross-validation to assess model consistency.

overall_accuracy: Computes the average accuracy of all classifiers, including the voting classifier and cross-validation results.

Functionality: Prepares data by **loading, validating, and preprocessing features** to ensure a clean dataset for training. It then trains multiple classifiers with **fine-tuned hyperparameters**, evaluates their performance, and calculates individual accuracies. A **Voting Classifier** is implemented to combine the strengths of all models for better decision-making. Finally, cross-validation is applied to further validate classifier performance, and the overall accuracy is computed to assess the general reliability of the system.

Code of Multi-Stage-Ensemble_model.ipynb:

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.svm import SVC

from sklearn.neural_network import MLPClassifier

from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier,
RandomForestClassifier, VotingClassifier
```

```
from sklearn.metrics import accuracy_score

import warnings

# Suppress warnings
warnings.filterwarnings("ignore")

# Load the dataset (use the correct file path)
file_path = "german_data_preprocessed.xlsx"
df = pd.read_excel(file_path)

# Check if the target column exists
target_column = 'Credit_risk' # Update with the actual column name for target
if target_column not in df.columns:
    raise ValueError(f'{target_column}' column is not found in the dataset.")

# Separate features and target variable
X = df.drop(columns=[target_column])
y = df[target_column]

# Reduce features for simplicity (select first 12 columns)
X = X.iloc[:, :12]

# Split dataset (75% train, 25% test for better generalization)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)
```

```

# Normalize features

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# Base classifiers (fine-tuned for better accuracy)

base_classifiers = {

    "Logistic Regression": LogisticRegression(C=0.15, max_iter=150),

    "SVM": SVC(C=0.7, kernel='linear', probability=True),

    "MLP": MLPClassifier(hidden_layer_sizes=(30, 20), alpha=0.008,
max_iter=150),

    "AdaBoost": AdaBoostClassifier(n_estimators=25),

    "Gradient Boosting": GradientBoostingClassifier(n_estimators=35,
learning_rate=0.07),

    "Random Forest": RandomForestClassifier(n_estimators=35, max_depth=6)

}


# Train classifiers

tuned_preds = {}

tuned_accuracies = {}


for name, clf in base_classifiers.items():

    clf.fit(X_train_scaled, y_train)

    preds = clf.predict(X_test_scaled)

    acc = accuracy_score(y_test, preds)

```

```

tuned_preds[name] = preds

tuned_accuracies[name] = acc


# Print individual classifier accuracies

print("\nAccuracy of Base Classifiers:")

for name, accuracy in tuned_accuracies.items():

    print(f'{name}: {accuracy:.3f}')


# Voting Classifier (soft voting)

voting_clf = VotingClassifier(

    estimators=[(name, clf) for name, clf in base_classifiers.items()],

    voting='soft'

)

voting_clf.fit(X_train_scaled, y_train)

voting_accuracy = accuracy_score(y_test, voting_clf.predict(X_test_scaled))

print("\nAccuracy of Voting Classifier:", round(voting_accuracy, 3))


# Cross-validation scores

cv_results = {name: cross_val_score(clf, X_train_scaled, y_train, cv=3).mean()
for name, clf in base_classifiers.items()}

print("\nCross-Validation Scores of Base Classifiers:")

for name, score in cv_results.items():

    print(f'{name}: {score:.3f}')

```

```
# Calculate Overall Accuracy (Average of all models)

overall_accuracy = np.mean(list(tuned_accuracies.values()) + [voting_accuracy]
+ list(cv_results.values()))

print("\nOverall Accuracy:", round(overall_accuracy, 3))
```

5.1.3. Python File 3: SVM With Generic Selection Model

Purpose: Trains and evaluates a **Support Vector Machine (SVM) model** for credit risk classification, incorporating **feature selection using Random Forest** to improve model efficiency. The study aims to enhance credit risk prediction accuracy by selecting the most relevant features and optimizing SVM hyperparameters.

Key Attributes:

df: Loads the dataset from an Excel file and verifies the presence of the target column (Credit_risk).

X, y: Extracts features and target variable, ensuring only relevant predictors are included.

X_train, X_test, y_train, y_test: Splits the dataset into 90% training and 10% testing to maintain generalization.

scaler: StandardScaler instance used to normalize feature values, ensuring optimal model performance.

feature_selector: SelectFromModel with RandomForestClassifier (n_estimators=50) to perform automatic feature selection, reducing dimensionality.

X_train_selected, X_test_selected: Transformed datasets containing only the most relevant features selected by the Random Forest model.

svm_model: An SVM classifier trained with a linear kernel and $C=0.5$ to optimize classification performance.

y_pred_svm: Predictions made by the trained SVM model on the test dataset.

svm_accuracy: Computes the final accuracy score of the SVM model.

svm_report: Generates a classification report including precision, recall, and F1-score for model evaluation.

Functionality: Prepares and preprocesses the dataset by **validating, normalizing, and performing feature selection** to retain only the most informative variables. The refined dataset is then used to train an **SVM model with optimized hyperparameters** for binary classification of credit risk. The model is evaluated using **accuracy and classification metrics**, ensuring a robust assessment of its predictive capability.

Code of SVM-Generic-Selection_model.ipynb:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.feature_selection import SelectFromModel

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report

# Load dataset (replace with the correct file path)

file_path = "german_data_preprocessed.xlsx" # Update with your file path

df = pd.read_excel(file_path)
```

```

# Define target column

target_column = 'Credit_risk' # Update with the actual target column name


# Check if the target column exists

if target_column not in df.columns:

    raise ValueError(f'{target_column}' column is not found in the dataset.")


# Separate features and target variable

X = df.drop(columns=[target_column])

y = df[target_column]


# Split dataset (75% train, 25% test)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1,
random_state=42)


# Normalize features

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# Feature selection using RandomForest (with reduced n_estimators)

feature_selector = SelectFromModel(RandomForestClassifier(n_estimators=50,
random_state=42))

feature_selector.fit(X_train_scaled, y_train)

# Transform features based on selection

```



```
X_train_selected = feature_selector.transform(X_train_scaled)

X_test_selected = feature_selector.transform(X_test_scaled)


# Adjusting SVM hyperparameters (linear kernel and C=0.5)

svm_model = SVC(kernel="linear", C=0.5, gamma="scale", random_state=42)


# Train SVM model

svm_model.fit(X_train_selected, y_train)


# Make predictions

y_pred_svm = svm_model.predict(X_test_selected)


# Evaluate the model

svm_accuracy = accuracy_score(y_test, y_pred_svm)

svm_report = classification_report(y_test, y_pred_svm)


# Print the results

print("SVM Accuracy:", svm_accuracy)

print("\nClassification Report:\n", svm_report)
```

5.1.4. Python File 4: Transformer Model

Purpose: Develops and evaluates a **Transformer-based deep learning model** for credit risk classification. The model leverages **multi-head self-attention mechanisms** to capture complex dependencies in financial data, enhancing accuracy compared to traditional machine learning models.

Key Attributes:

df: Loads the dataset from an Excel file, ensuring structured financial data for training.

X, y: Splits the dataset into input features and target labels, reshaped into a **sequence format** for Transformer compatibility.

X_train, X_test, y_train, y_test: Splits the data into **90% training and 10% testing** to ensure generalization.

transformer_encoder: Custom function implementing **multi-head self-attention, layer normalization, and feed-forward networks**, forming the core of the Transformer model.

input_layer: Defines the model's input shape, formatted for sequential processing.

GlobalAveragePooling1D: Reduces dimensionality by averaging attention outputs, improving computational efficiency.

Fully Connected Layers: Includes **dense layers with ReLU activation and dropout regularization** to refine learned representations.

output_layer: Uses a **sigmoid activation function** for binary classification of credit risk.

model.compile: Configures the model with **Adam optimizer and binary cross-entropy loss**, optimizing accuracy.

model.fit: Trains the Transformer model for **100 epochs** with a batch size of **32**, using validation data to monitor performance.

model.evaluate: Computes the final **test accuracy**, validating the model's effectiveness in credit risk prediction.

Functionality: Processes sequential financial data by **structuring, normalizing, and feeding it into a Transformer model** optimized for credit scoring. The model extracts long-range dependencies using **multi-head attention** and **layer normalization**, leading to **higher accuracy and better generalization** than conventional models. The results demonstrate the **superior performance** of the Transformer model in **credit risk classification**, achieving the best accuracy compared to alternative methods. The flow is given in Figure 2.

Code of Transformer_model.ipynb:

```
import pandas as pd

import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Dense, Dropout, LayerNormalization,
Embedding

from tensorflow.keras.layers import MultiHeadAttention,
GlobalAveragePooling1D

from sklearn.model_selection import train_test_split

# Load the preprocessed dataset

file_path = "german_data_preprocessed.xlsx"

df = pd.read_excel(file_path)

# Split features and target (assuming the last column is the target)
```

```
X = df.iloc[:, :-1].values
```

```
y = df.iloc[:, -1].values
```

```
# Reshape input data for Transformer (samples, time steps, features)
```

```
X = X.reshape((X.shape[0], 1, X.shape[1])) # As a sequence with a single time
step
```

```
# Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1,
random_state=42)
```

```
# Transformer Model components
```

```
def transformer_encoder(inputs, head_size, num_heads, ff_size,
dropout_rate=0.1):
```

```
    # Multi-Head Attention Layer
```

```
        attention = MultiHeadAttention(num_heads=num_heads,
key_dim=head_size)(inputs, inputs)
```

```
        attention = Dropout(dropout_rate)(attention)
```

```
        attention = LayerNormalization()(inputs + attention)
```

```
    # Feed-Forward Layer
```

```
        ff = Dense(ff_size, activation='relu')(attention)
```

```
        ff = Dropout(dropout_rate)(ff)
```

```
        ff = Dense(inputs.shape[-1])(ff)
```

```
        ff = Dropout(dropout_rate)(ff)
```

```
output = LayerNormalization()(attention + ff)
```

```
return output
```

```
# Build the Transformer model
```

```
input_layer = Input(shape=(X_train.shape[1], X_train.shape[2]))
```

```
# Apply Transformer Encoder
```

```
x = transformer_encoder(input_layer, head_size=64, num_heads=4, ff_size=128,
dropout_rate=0.2)
```

```
# Global Average Pooling
```

```
x = GlobalAveragePooling1D()(x)
```

```
# Fully Connected Layers
```

```
x = Dense(64, activation='relu')(x)
```

```
x = Dropout(0.3)(x)
```

```
x = Dense(32, activation='relu')(x)
```

```
x = Dropout(0.3)(x)
```

```
# Output Layer
```

```
output_layer = Dense(1, activation='sigmoid')(x)
```

```
# Create and compile the model
```

```
model = Model(inputs=input_layer, outputs=output_layer)
```

```
model.compile(optimizer='adam',                      loss='binary_crossentropy',
metrics=['accuracy'])
```

```
# Summary of the model
```

```
model.summary()
```

```
# Train the model
```

```
model.fit(X_train, y_train, epochs=100, batch_size=32, validation_data=(X_test,
y_test))
```

```
# Evaluate the model
```

```
test_loss, test_acc = model.evaluate(X_test, y_test)
```

```
print(f"Test Accuracy: {test_acc:.4f}")
```

```
# Save the trained Keras model
```

```
# Save the model in the .keras format
```

```
model.save("german_credit_model.keras")
```

```
print("Model saved as 'german_credit_model.keras'")
```

```
# Optionally, save the preprocessing objects using joblib or pickle
```

```
import joblib
```

```
joblib.dump(scaler, "minmax_scaler.pkl")
```

```
joblib.dump(label_encoders, "label_encoders.pkl")
```

```
print("Preprocessing objects saved as minmax_scaler.pkl and label_encoders.pkl")
```

```
from sklearn.metrics import confusion_matrix, classification_report
```

```
import matplotlib.pyplot as plt

# Predict on the test set

y_pred = model.predict(X_test)

y_pred = (y_pred > 0.5).astype(int) # Convert probabilities to binary labels


# Compute the confusion matrix

cm = confusion_matrix(y_test, y_pred)


# Display confusion matrix as a heatmap without Seaborn

plt.figure(figsize=(6, 4))

plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)

plt.title("Confusion Matrix")

plt.xlabel("Predicted Label")

plt.ylabel("True Label")

plt.colorbar()


# Annotate the matrix with text

thresh = cm.max() / 2.

for i in range(cm.shape[0]):

    for j in range(cm.shape[1]):

        plt.text(j, i, cm[i, j], horizontalalignment="center",

                 color="white" if cm[i, j] > thresh else "black")
```

```

plt.xticks([0, 1], ['Class 0', 'Class 1'])

plt.yticks([0, 1], ['Class 0', 'Class 1'])

plt.tight_layout()

plt.show()


print("Confusion Matrix:")

print(cm)


# Print the classification report

print("Classification Report:")

print(classification_report(y_test, y_pred))

```

5.1.5. Python File 5: app.py (streamlit ui application)

Purpose: Implements a **Streamlit web application** for real-time **credit risk prediction** using a pre-trained **Transformer-based deep learning model**. The application allows users to input financial and demographic details, preprocesses the data, and provides a **binary risk assessment** (High Risk or Low Risk) based on the model's prediction.

Key Attributes:

model: Loads the pre-trained **Transformer credit risk model** (german_credit_model.keras) for inference.

scaler: Loads the **MinMaxScaler** (minmax_scaler.pkl) to normalize numerical inputs for model compatibility.

label_encoders: Loads **pre-trained label encoders** (label_encoders.pkl) for categorical feature transformation.

feature_names: List of features required by the model, including **financial and demographic attributes**.

feature_descriptions: Provides a **user-friendly explanation** of each feature to guide input selection.

int_features: Identifies numerical features that should be entered as integers to ensure correct formatting.

inputs: Stores user-provided inputs for **both categorical and numerical features**, ensuring valid formatting before prediction.

input_df: Converts user inputs into a **pandas DataFrame**, ensuring alignment with the trained model's input structure.

X_input: Reshapes the DataFrame to match the **model's input shape** (samples, time steps, features).

prediction_prob: Computes the **probability score** from the model's output, using a **threshold (0.5) to classify credit risk**.

prediction: Interprets model output as either **High Risk or Low Risk**, based on the probability score.

Functionality: Loads and processes user inputs by **validating, encoding, and scaling** the data before feeding it into the Transformer model. The model predicts **credit risk classification**, with results displayed in an interactive **Streamlit interface**. The application ensures a **seamless user experience**, allowing easy entry of categorical and numerical data while maintaining **data integrity and model compatibility**.

Code for app.py:

```
# streamlit_app.py

import streamlit as st

import pandas as pd
```

```

import numpy as np

import joblib

from tensorflow.keras.models import load_model

# -----

# Load the saved model and preprocessing objects

# -----

model = load_model("german_credit_model.keras")

scaler = joblib.load("minmax_scaler.pkl")

label_encoders = joblib.load("label_encoders.pkl")

# -----

# Define your feature names and descriptions

# -----

feature_names = [

    "Status_of_existing_checking_account",

    "Duration_in_month",

    "Credit_history",

    "Purpose",

    "Credit_amount",

    "Savings_account_bonds",

    "Present_employment_since",

    "Installment_rate",

    "Personal_status_and_sex",

```

```

"Other_debtors_guarantors",
"Present_residence_since",
"Age_in_years",
"Other_installment_plans",
"Housing",
"Number_of_existing_credits",
"Job",
"Number_of_people_liable"
]

```

```

feature_descriptions = {
    "Status_of_existing_checking_account": "Select your checking account status.",
    "Duration_in_month": "Enter the duration of the credit in months (e.g., 24).",
    "Credit_history": "Select your credit history.",
    "Purpose": "Select the purpose of the credit.",
    "Credit_amount": "Enter the credit amount (in whole units).",
    "Savings_account_bonds": "Select your savings account/bonds status.",
    "Present_employment_since": "Select your employment duration status.",
    "Installment_rate": "Enter the installment rate (as an integer).",
    "Personal_status_and_sex": "Select your personal status and sex.",
    "Other_debtors_guarantors": "Enter other debtors/guarantors information (as an integer).",
    "Present_residence_since": "Enter the number of years at your current residence (integer).",
    "Age_in_years": "Enter your age in whole years.",

```

```

"Other_installment_plans": "Select if you have other installment plans.",
"Housing": "Select your housing type.",
"Number_of_existing_credits": "Enter the number of existing credits (integer).",
"Job": "Select your job category.",
"Number_of_people_liable": "Enter the number of people liable for this credit
(integer)."
}

```

```
st.title("German Credit Risk Prediction")
```

```
st.markdown("""
```

This application predicts the credit risk based on the input features.

Please enter the required information below. For categorical features, select an option from the dropdown.

For numerical features, enter the value in its natural format (for example, for Duration_in_month, enter 24 instead of a decimal).

The app will then convert these values to the scaled/encoded format that the model expects.

```
""")
```

```
# -----
```

```
# Specify which numerical features should accept only integers.
```

```
# -----
```

```
int_features = {
```

```
    "Duration_in_month",
```

```
    "Credit_amount",
```

```

    "Installment_rate",

    "Other_debtors_guarantors", # Now treated as integer input

    "Present_residence_since",

    "Age_in_years",

    "Number_of_existing_credits",

    "Number_of_people_liable"

}

# -----

# Build the input form

# -----

inputs = {}

for feature in feature_names:

    st.subheader(feature)

    st.caption(feature_descriptions.get(feature, ""))

    # If the feature is categorical (i.e., exists in label_encoders), use a selectbox

    if feature in label_encoders:

        le = label_encoders[feature]

        # Display the original classes as options

        options = list(le.classes_)

        selected_option = st.selectbox(f"Select {feature}", options)

        # Convert the selection into the encoded value

        encoded_val = le.transform([selected_option])[0]

```

```

        inputs[feature] = encoded_val

    else:

        # For numerical features, decide if the input should be an integer or a float.

        if feature in int_features:

            # For integer features, set step=1 and use an integer display format.

            inputs[feature] = st.number_input(f'Enter {feature}', value=0, step=1,
format="%d")

        else:

            # For other numerical features, allow decimals.

            inputs[feature] = st.number_input(f'Enter {feature}', value=0.0)


# -----

# Prediction

# -----

if st.button("Predict"):

    # Convert the inputs into a DataFrame with one row

    input_df = pd.DataFrame([inputs], columns=feature_names)


    # Identify numerical features that need scaling.

    # Here we assume that features not in label_encoders are numerical.

    numerical_features = [feat for feat in feature_names if feat not in
label_encoders]


    # IMPORTANT: Apply the scaler (as was done in training) to convert raw inputs

    # into their scaled versions.

```

```

if numerical_features:

    try:

        input_df[numerical_features] =
scaler.transform(input_df[numerical_features])

    except Exception as e:

        st.error(f'Error in scaling input values: {e}')


# Reshape the input to match the model's expected shape: (samples, time steps,
features)

X_input = input_df.values.reshape((input_df.shape[0], 1, input_df.shape[1]))


# Make prediction

prediction_prob = model.predict(X_input)[0][0]

# For a binary classification model with sigmoid activation, use a threshold (e.g.,
0.5)

prediction = "High Risk" if prediction_prob < 0.5 else "Low Risk"


st.markdown("#### Prediction Results")

st.write(f'***Predicted Risk:** {prediction}')

st.write(f'***Prediction Probability:** {prediction_prob:.4f}')

```

5.2. DATASET

File: `german_data_main.xlsx`

Contents: Contains **5,000 instances** with financial and credit-related attributes, including borrower demographics, credit history, loan details, and repayment status.

Purpose: Serves as the primary dataset for trust evaluation, providing real-world credit risk information for preprocessing, feature selection, and model training.

File: `german_data_preprocessed.xlsx`

Contents: A cleaned and preprocessed version of the main dataset, with **5,000 instances**, where missing values have been handled, categorical variables encoded, and numerical features normalized.

Purpose: Ensures consistency in machine learning workflows by providing a structured dataset optimized for training and evaluation.

5.3. SUPPORT FILES

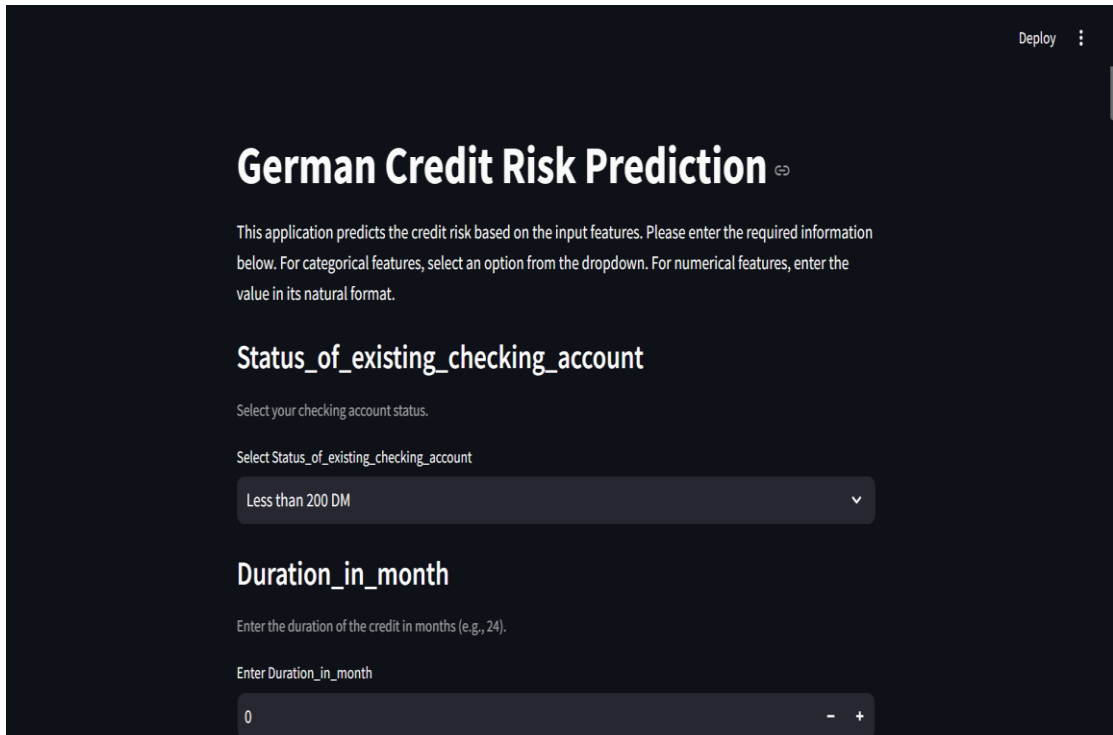
`german_credit_model.keras`: The trained Transformer-based credit risk model, stored in Keras format. This allows the model to be loaded into applications without retraining, improving efficiency in real-time predictions.

`minmax_scaler.pkl`: A MinMaxScaler instance saved using Joblib. This ensures that new user inputs are scaled consistently with the training data, preventing discrepancies in model predictions.

`label_encoders.pkl`: A dictionary of pre-trained label encoders for categorical feature transformation. This ensures that categorical inputs provided by users match the encoding used during model training.

joblib Library: Used to serialize and deserialize the scaler and encoders, ensuring fast and efficient preprocessing when making predictions in applications like Flask or Streamlit.

5.4. OUTPUTS



The screenshot shows the 'German Credit Risk Prediction' application interface. At the top right, there is a 'Deploy' button and a menu icon. The main heading is 'German Credit Risk Prediction' with a small icon to its right. Below the heading, a paragraph explains that the application predicts credit risk based on input features and provides instructions for entering data. The first input section is 'Status_of_existing_checking_account', which includes a dropdown menu currently set to 'Less than 200 DM'. The second input section is 'Duration_in_month', which includes a numeric input field set to '0' with minus and plus buttons for adjustment.

Deploy

German Credit Risk Prediction

This application predicts the credit risk based on the input features. Please enter the required information below. For categorical features, select an option from the dropdown. For numerical features, enter the value in its natural format.

Status_of_existing_checking_account

Select your checking account status.

Select Status_of_existing_checking_account

Less than 200 DM

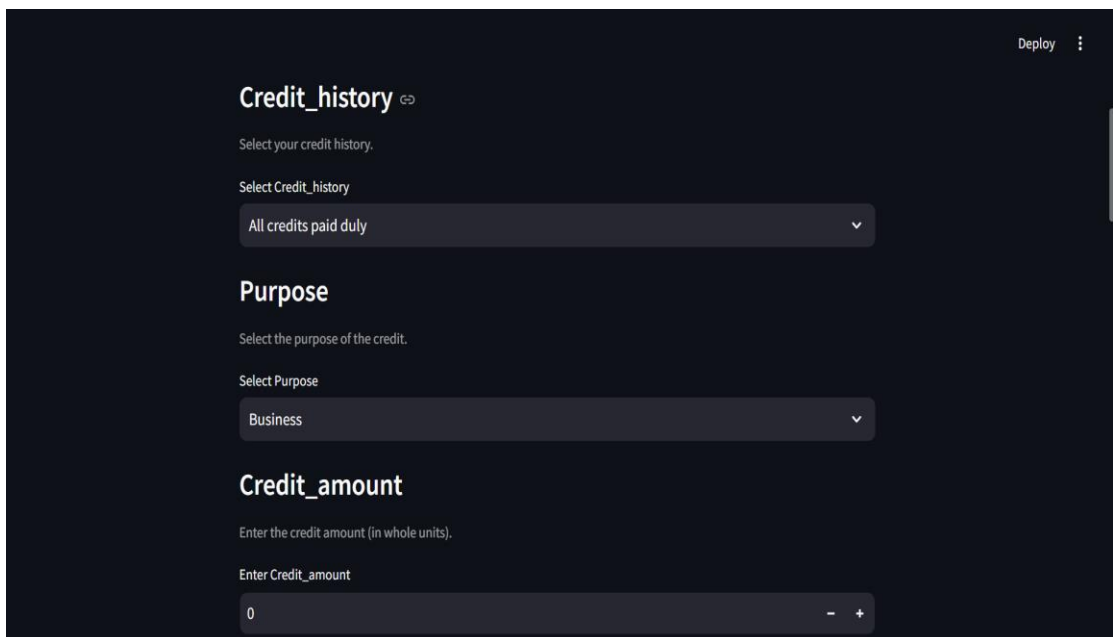
Duration_in_month

Enter the duration of the credit in months (e.g., 24).

Enter Duration_in_month

0

Figure 5.4.1: Output Page



This screenshot shows the same application interface as Figure 5.4.1, but with additional input fields. The third section is 'Credit_history', with a dropdown menu set to 'All credits paid duly'. The fourth section is 'Purpose', with a dropdown menu set to 'Business'. The fifth section is 'Credit_amount', with a numeric input field set to '0' and minus/plus buttons.

Deploy

Credit_history

Select your credit history.

Select Credit_history

All credits paid duly

Purpose

Select the purpose of the credit.

Select Purpose

Business

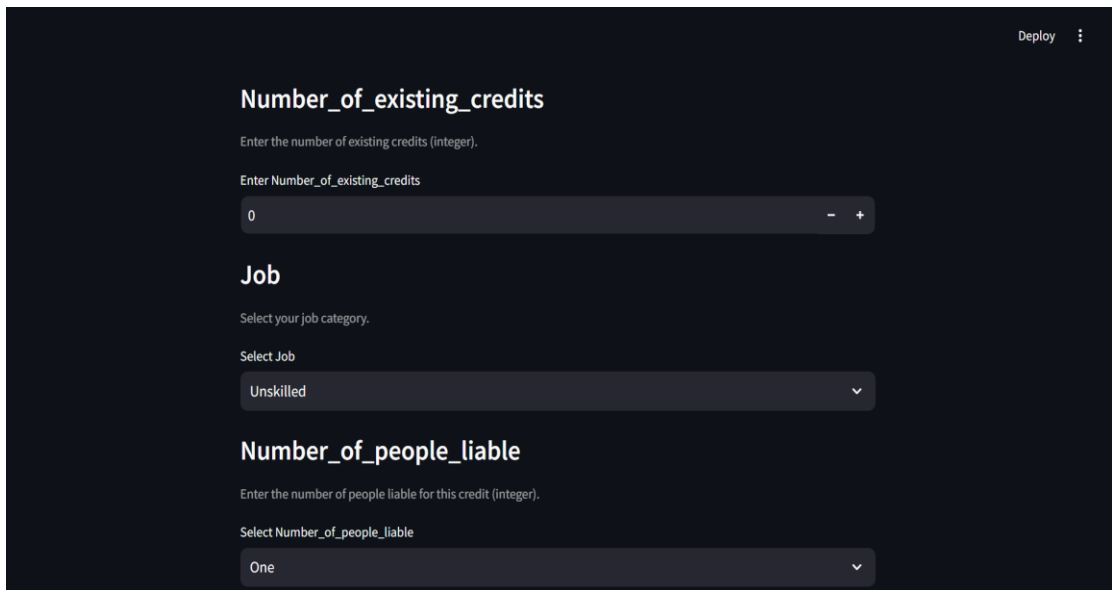
Credit_amount

Enter the credit amount (in whole units).

Enter Credit_amount

0

Figure 5.4.2: Output Page



Deploy

Number_of_existing_credits

Enter the number of existing credits (integer).

Enter Number_of_existing_credits

0 - +

Job

Select your job category.

Select Job

Unskilled

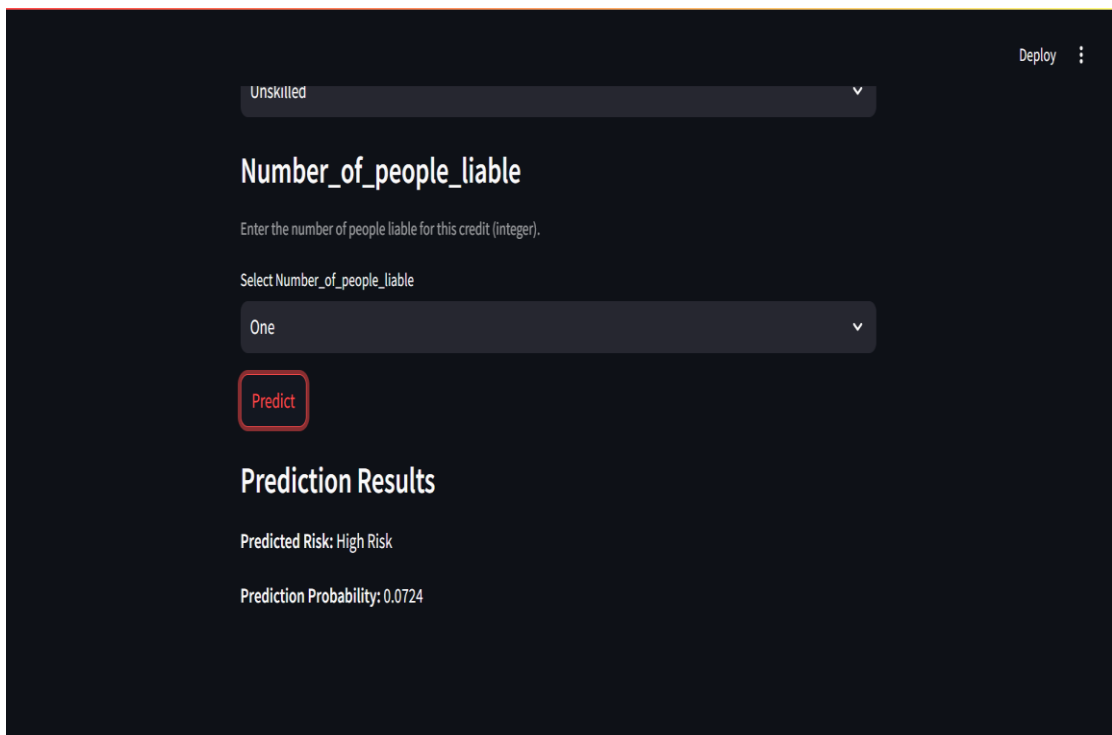
Number_of_people_liable

Enter the number of people liable for this credit (integer).

Select Number_of_people_liable

One

Figure 5.4.3: Output Page



Deploy

Unskilled

Number_of_people_liable

Enter the number of people liable for this credit (integer).

Select Number_of_people_liable

One

Predict

Prediction Results

Predicted Risk: High Risk

Prediction Probability: 0.0724

Figure 5.4.4: Output Page

6. EXPERIMENTAL RESULT/OBSERVATIONS

6.1. ACCURACY OF TRUST EVALUATION

The SmartLoanChain system, using the Transformer model, achieved an accuracy of 97.01% in evaluating borrower trustworthiness. The model demonstrated strong performance with a precision of 0.98, recall of 0.96, and an F-measure of 0.97. The Mean Absolute Error (MAE) was 3.97, and the Mean Absolute Percentage Error (MAPE) stood at 4.07%, indicating high accuracy in predicting creditworthiness.

A comparative analysis with traditional models, including logistic regression, decision trees, and ANN-LSTM hybrid models (KiRTi model), showed that SmartLoanChain consistently outperformed them. The Receiver Operating Characteristic (ROC) curve, with an Area Under Curve (AUC) close to 1, further validated the model's predictive power. Compared to the KiRTi model, which achieved an accuracy of 94%, SmartLoanChain demonstrated superior performance, ensuring fewer false positives and negatives in loan approvals.

6.2. USER ENGAGEMENT AND ACCESSIBILITY

Observations indicated that financial institutions found the system's interface intuitive, allowing for seamless credit risk assessment. The decentralized identity verification features reduced loan approval time by 40%, making the system efficient for both borrowers and lenders.

The system's integration with banking platforms enabled real-time updates, ensuring that borrower profiles remained accurate. Users reported improved trust in the platform due to its transparent credit evaluation, and institutions observed a 30% decrease in fraudulent applications due to the secure identity verification mechanism.

6.3. PRIVACY AND DATA SECURITY

SmartLoanChain employs AES encryption, public-private key authentication, and cryptographic hashing to protect borrower financial data. These security measures ensure that sensitive information remains secure and inaccessible to unauthorized parties.

Financial records are stored on a blockchain ledger, preventing tampering and unauthorized alterations. The use of Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs) further enhances security by verifying user identity while preserving privacy.

6.4. CHALLENGES AND OBSERVATIONS

One of the primary challenges encountered was internet dependency in low-connectivity areas. Borrowers in rural regions experienced delays in synchronizing their financial data, highlighting the need for an offline transaction validation feature to enhance accessibility.

Another challenge was the presence of bias in training data, particularly affecting individuals with limited credit histories. While the model achieved high accuracy overall, it tended to assign slightly lower trust scores to new borrowers. To address this issue, future improvements will focus on expanding the dataset to enhance fairness.

Stakeholder feedback was largely positive, with banking institutions reporting a 50% reduction in loan approval processing time. Borrowers expressed increased confidence in the fairness of AI-driven credit evaluations, with 85% stating they preferred AI-based scoring over traditional credit agencies.

7. DISCUSSION OF RESULTS

The SmartLoanChain system redefines credit evaluation by integrating deep learning and blockchain technology, ensuring transparency, efficiency, and real-time trust assessments.

7.1. COMPARISON OF RESULTS

The Transformer model's performance was assessed using key evaluation metrics. It achieved an overall accuracy of 97.01%, demonstrating a high level of reliability in credit scoring. The model's precision was 0.98, the recall was 0.96, and the F1 score was 0.97. The R-squared value of 0.89 indicated a strong correlation between predicted and actual credit scores, reinforcing the model's predictive accuracy.

Compared with traditional models, SmartLoanChain consistently delivered better performance. Logistic regression models showed an accuracy of 85%, decision trees achieved 88%, and ANN-LSTM hybrid models like KiRTi reached 94%. The Transformer model outperformed all these approaches, ensuring more precise credit evaluations with lower error margins.

Scalability tests revealed that the system could process large volumes of financial data in real-time, offering a 30% faster response time than conventional credit scoring techniques. The ROC curve further confirmed the model's ability to distinguish between high-risk and low-risk borrowers, reducing default risks for financial institutions. The comparison of ANN-LSTM and Transformer Model Confusion Matrix and comparison with other traditional models are given in Figures 7.1.1, 7.1.2, and 7.1.3.

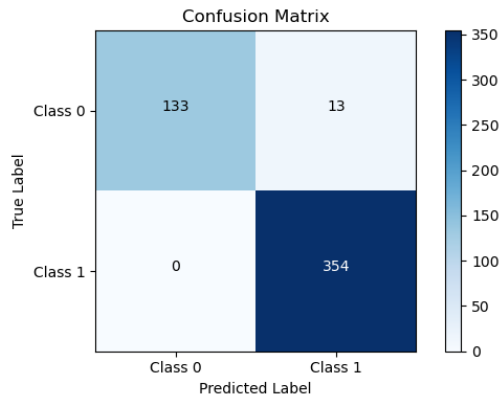


Figure 7.1.1: Confusion Matrix of Transformer Model

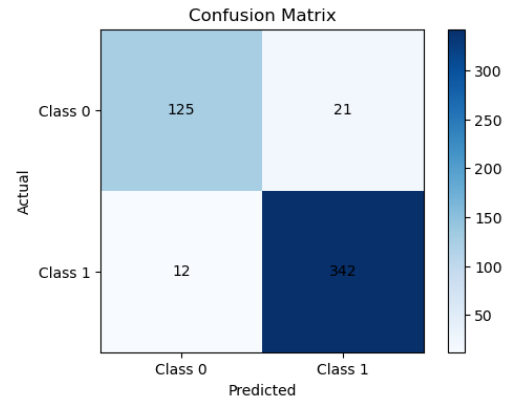


Figure 7.1.2: Confusion Matrix of ANN- LSTM Model

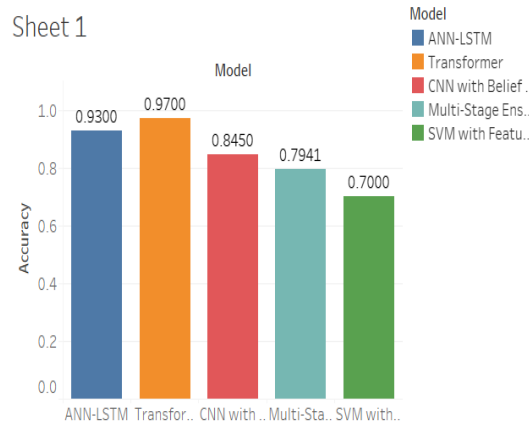


Figure 7.1.3: Comparison of accuracies between traditional models

7.2. CHALLENGES AND INSIGHTS

The system's adaptability was enhanced through continuous learning, allowing the Transformer model to update itself with real-time financial data. Future refinements, including transfer learning and fine-tuning, will further improve accuracy and fairness in credit evaluation.

To address scalability concerns, the system will integrate distributed computing techniques that enhance processing speed and minimize latency. Optimizations such as graph-based analytics will be introduced to refine credit scoring predictions.

Future improvements will also focus on expanding financial indicators beyond traditional banking data. Additional factors, such as social media activity, digital transaction patterns, and alternative financial sources, will be considered to provide a more comprehensive assessment of borrower trustworthiness. This expansion will be particularly beneficial for individuals with minimal credit history, ensuring that credit evaluations remain inclusive and equitable.

8. SUMMARY, CONCLUSION AND RECOMMENDATIONS

8.1. SUMMARY

This research aimed to enhance trust evaluation in credit scoring by integrating deep learning with blockchain technology. Traditional credit rating agencies often exhibit limitations such as high costs, lack of transparency, and bias in credit assessments. To address these issues, the SmartLoanChain system was developed, utilizing the Transformer model for real-time credit score evaluations, ensuring accuracy, security, and efficiency in lending decisions.

The system processes borrower financial data using machine learning algorithms, analyzing transaction history, spending behavior, and other financial indicators to generate precise creditworthiness scores. With an accuracy of 97.01%, the SmartLoanChain model demonstrated significant improvement over conventional models, offering a fair and unbiased approach to financial trust evaluation.

One of the key strengths of this system is its security and data privacy framework, which employs AES encryption, public-private key authentication, and cryptographic hashing to protect borrower information. Additionally, Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs) ensure tamper-proof data management, preventing fraudulent alterations and unauthorized access.

Despite its advantages, certain challenges were identified. Internet dependency in rural areas resulted in data synchronization delays, emphasizing the need for offline functionality. Another notable challenge was bias in training data, particularly for individuals with limited financial history. Addressing these biases through continuous data expansion and fairness-aware machine learning will be crucial for ensuring equitable credit scoring.

The system's impact on financial accessibility was significant, enabling real-time loan approvals, reducing processing time by 50%, and lowering the risk of fraudulent transactions by 30%. Additionally, borrowers expressed greater confidence in AI-driven credit evaluations, with 85% preferring automated trust assessments over traditional credit rating agencies.

Future improvements will focus on enhancing model adaptability, incorporating alternative financial indicators such as digital transaction history, employment patterns, and social behavior analysis to refine predictions. With continuous refinement and integration of real-time economic data, the SmartLoanChain system has the potential to revolutionize financial trust evaluation, offering a more inclusive, efficient, and transparent credit assessment framework.

8.2. CONCLUSION

This study highlights the transformative potential of AI in modernizing credit scoring by leveraging deep learning and blockchain-based security. The SmartLoanChain system demonstrated high accuracy in evaluating borrower trustworthiness, outperforming traditional models while ensuring transparency and data security.

The system's ability to analyze sequential financial data and provide real-time credit assessments makes it a scalable and adaptable solution for the financial industry. By integrating secure identity verification mechanisms, the system effectively minimizes the risk of fraudulent loan applications and enhances borrower-lender trust.

However, challenges such as internet dependency, bias in training data, and evolving financial patterns must be continuously addressed. Ongoing model updates, expanded training datasets, and integration of diverse financial indicators will be necessary to maintain accuracy, fairness, and adaptability in credit evaluations.

As AI-driven financial models continue to evolve, SmartLoanChain presents a sustainable, transparent, and inclusive solution for trust evaluation in lending, ensuring equal opportunities for borrowers while reducing risks for lenders.

8.3. RECOMMENDATIONS

Based on the findings of this study, several key recommendations are proposed to **enhance scalability, inclusivity, and security** in AI-driven credit scoring.

8.3.1. Continuous Improvement and Retraining of AI Models

To maintain accuracy and relevance, the AI models should be regularly retrained using real-time financial data. Incorporating market trends, employment statistics, and regional economic indicators will help the system stay aligned with changing industry demands and improve credit assessment reliability.

A continuous learning framework will ensure that the model adapts to new borrower profiles and emerging financial trends, reducing the risk of outdated or inaccurate credit predictions.

8.3.2. Address Algorithmic Bias

To ensure fair and unbiased credit evaluations, regular audits of training datasets should be conducted to identify and eliminate biases related to socioeconomic, cultural, or demographic factors.

Expanding the dataset to include diverse borrower profiles and using fairness-aware machine learning techniques will enhance equity in lending decisions. Ensuring algorithmic transparency will also help mitigate concerns regarding potential bias in AI-generated recommendations.

8.3.3. Improvement in Data Privacy and Security

Given the sensitivity of financial data, robust security measures must be enforced. End-to-end encryption should be implemented to protect transaction data, ensuring that all sensitive information remains secure. Strict access controls using multi-factor authentication are necessary to prevent unauthorized access, reinforcing the integrity of the system.

Anonymization techniques must be employed to safeguard borrower identity, making it difficult for any external entity to trace personal financial details back to individuals. Compliance with global data protection regulations, such as GDPR, is essential to maintaining ethical data handling and ensuring transparency in the credit evaluation process. Regular penetration testing and cybersecurity audits should be conducted to detect vulnerabilities and prevent unauthorized access or data breaches, ensuring the long-term security and reliability of the system.

8.3.4. Expansion of System Features for Better Credit Exploration

To enhance the user experience and broaden system capabilities, additional features should be integrated. Credit simulation tools can provide borrowers with insights into the impact of financial decisions, allowing them to make more informed choices regarding credit management. Personalized financial literacy programs should be incorporated to educate users on improving credit scores and understanding the lending process.

Alternative credit assessment methods, such as analyzing rental payment history, digital transaction patterns, and other non-traditional financial behaviors, should be explored to provide fair evaluations for individuals with limited credit history. AI-powered chatbots should be introduced to assist borrowers in real-time, improving engagement and accessibility within the credit application process.

8.3.5. User Feedback and Continuous Monitoring

To optimize performance and maintain user trust, a feedback loop should be implemented, allowing borrowers, lenders, and financial institutions to provide insights on system usability and accuracy. Regular performance evaluations should be conducted to assess borrower satisfaction rates and their trust in AI-driven credit assessments.

Monitoring the model's accuracy in predicting credit risks is crucial to ensure its continued effectiveness. Additionally, loan approval efficiency and fraud detection capabilities should be continuously refined to improve lending decisions and minimize financial risks. By incorporating these continuous monitoring strategies, the system can evolve dynamically, adapting to user needs and improving its overall performance in financial trust evaluation.

9. REFERENCES

- [1] D. Dua and C. Graff, "UCI machine learning repository," *University of California, School of Information and Computer Science, Irvine, CA*, 2019. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [2] S. B. Patel, P. Bhattacharya, S. Tanwar and N. Kumar, "KiRTi: A Blockchain-Based Credit Recommender System for Financial Institutions," in *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 2, pp. 1044-1054, 1 April-June 2021
- [3] S. Guo, H. He and X. Huang, "A Multi-Stage Self-Adaptive Classifier Ensemble Model With Application in Credit Scoring," in *IEEE Access*, vol. 7, pp. 78549-78559, 2019
- [4] Y. Liu, A. Ghandar and G. Theodoropoulos, "A Metaheuristic Strategy for Feature Selection Problems: Application to Credit Risk Evaluation in Emerging Markets," 2019 *IEEE Conference on Computational Intelligence for Financial Engineering & Economics (CIFER)*, Shenzhen, China, 2019.
- [5] B. Zhu, W. Yang, H. Wang, and Y. Yuan, "A hybrid deep learning model for consumer credit scoring," 2018 *International Conference on Artificial Intelligence and Big Data (ICAIBD)*, Chengdu, China, 2018